

GreenBite

Analisis de Patrones de Diseno y Arquetipos Arquitectonicos

1. Contexto del sistema

GreenBite es una plataforma B2C de suscripcion de alimentos organicos. Permite a usuarios suscribirse semanalmente a cajas con productos de agricultores locales. El sistema fue disenado con una arquitectura de microservicios para soportar la separacion de responsabilidades, la escalabilidad independiente de cada componente y la facilidad de mantenimiento a largo plazo.

2. Patrones de Diseno implementados

2.1 Patron Facade

Proposito: Proporcionar una interfaz simplificada a un conjunto complejo de subsistemas.

Implementacion en GreenBite: La clase `BackendParaFrontendService` actua como Facade. El frontend React no necesita saber que existen dos microservicios distintos. Solo se comunica con un unico punto de entrada (puerto 8080) y el Facade decide internamente si debe consultar al `producto-service` o al `compra-service`.

Beneficio concreto: Si en el futuro se agrega un tercer microservicio (por ejemplo, de usuarios o de pagos), el frontend no necesita cambios.

2.2 Patron Singleton

Proposito: Garantizar que una clase tenga una unica instancia compartida en toda la aplicacion.

Implementacion en GreenBite: Spring Boot gestiona todos los servicios anotados con `@Service` como singletons. Esto aplica a `BackendParaFrontendService`, `ProductoServicio` y `CompraServicio`. En ningun momento se instancian manualmente estos objetos; Spring los crea una sola vez y los reutiliza en cada peticion HTTP.

2.3 Patron Repository

Proposito: Separar la logica de negocio del acceso a la base de datos.

Implementacion en GreenBite: `ProductoRepository` y `CompraRepository` extienden `JpaRepository` y encapsulan todas las operaciones de base de datos. Los servicios (`ProductoServicio`, `CompraServicio`) no contienen sentencias SQL ni conocen el motor de base de datos utilizado.

2.4 Patron Builder

Proposito: Construir objetos complejos paso a paso, evitando constructores con muchos parametros.

Implementacion en GreenBite: La clase `Compra` implementa su propio Builder estatico. Esto permite crear suscripciones de forma clara y legible, especificando solo los campos necesarios sin depender del orden de los parametros.

```
Compra compra = new
Compra.Builder().items(lista).total(0.0).estado("PENDIENTE").build();
```

2.5 Patron Observer (frontend)

Proposito: Notificar a multiples componentes cuando el estado cambia.

Implementacion en GreenBite: En el frontend React, el estado del carrito se gestiona con useState en el componente App. Tanto CatalogoCajas como Carrito son observadores: cuando el usuario agrega una caja, el estado cambia y ambos componentes se actualizan automaticamente sin necesidad de comunicacion directa entre ellos.

3. Arquetipos Arquitectonicos

El sistema GreenBite utiliza el arquetipo Backend Para Frontend (BFF), que es un patron arquitectonico que coloca una capa de agregacion entre el cliente y los microservicios del backend.

Componente	Arquetipo	Responsabilidad
frontend	Cliente SPA	Interfaz de usuario en React
bff-service	Backend Para Frontend	Agrega y adapta datos para el frontend
producto-service	Microservicio	Gestion del catalogo de cajas organicas
compra-service	Microservicio	Registro de suscripciones y pedidos

4. Justificacion de las decisiones

Se eligio una arquitectura de microservicios porque GreenBite tendra componentes que escalan de forma diferente: el catalogo de productos es mas estable y de solo lectura, mientras que el servicio de compras recibira alta carga los dias de lunes (dia de entrega). Separarlos permite escalar solo el componente necesario.

El Backend Para Frontend se justifica porque el frontend React necesita datos combinados (informacion de la caja mas disponibilidad de stock) que provienen de distintos microservicios. Sin el BFF, el frontend tendria que hacer multiples llamadas y combinar los datos por su cuenta, lo que acopla el cliente con la arquitectura interna del backend.